

CPU Virtualization

Youngjae Kim (Ph.D.)

Sogang University

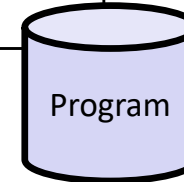
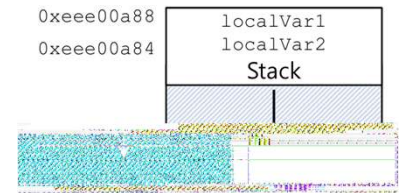
Linux 운영체제 및 응용 (GITF315-01)

Recap) Process

```

1 #include <stdio.h>
2 #include <unistd.h>
3 #include <assert.h>
4 #include <fcntl.h>
5 #include <sys/types.h>
6
7 int
8 main(int argc, char *argv[])
9 {
10     int fd = open("/tmp/file",
11                  O_WRONLY | O_CREAT | O_TRUNC, S_IRWXU);
12     assert(fd > -1);
13     int rc = write(fd, "hello world\n", 13);
14     assert(rc == 13);
15     close(fd);
16     return 0;
17 }

```



■ A process is a program in execution.

- A **program** is a passive entity — a file containing a list of instructions stored on disk (an executable file).
- A **process** is an active entity with its own program counter and runtime state.
- A program becomes a process when its executable file is loaded into memory.
 - One program can correspond to multiple processes.

■ Comprising of a process:

- Memory (address space)
 - Instructions, Data section
- Registers
 - Program counter, Stack pointer

<Process>

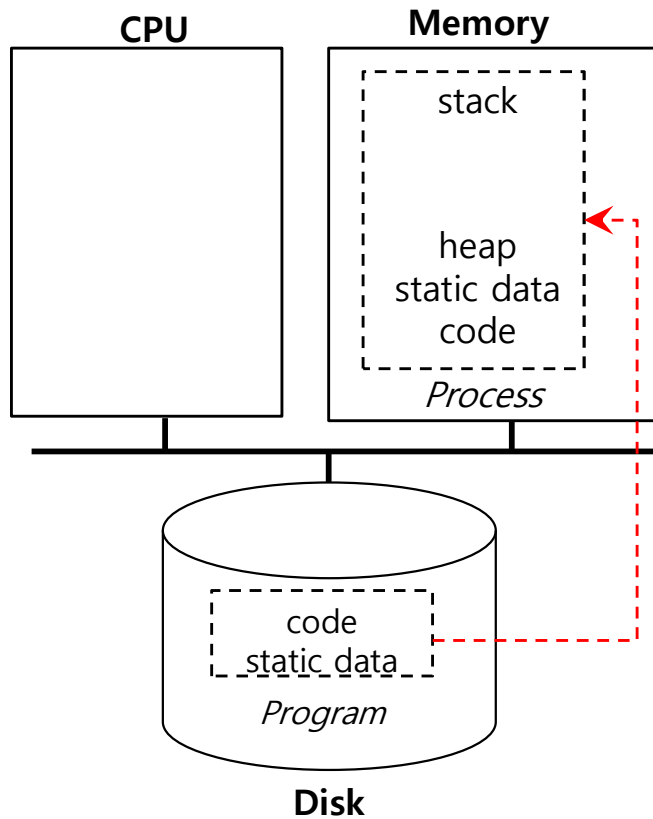
Main Memory

01100001	Instruction
01100010	Number
01100011	Instruction
01100100	Address
01100101	Instruction
01100111	Instruction
01101000	Address
01101001	Instruction
01101010	Address
01000111	

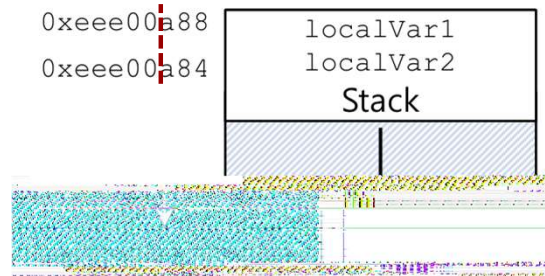


<Main Memory>

Recap) Process Creation



Loading:
Takes on-disk program
and reads it into the
address space of process



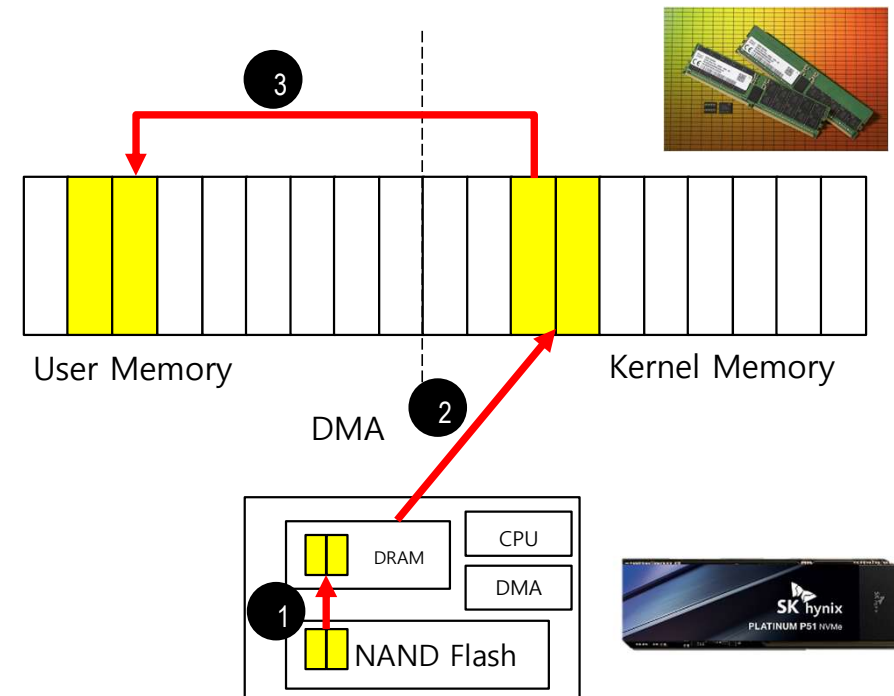
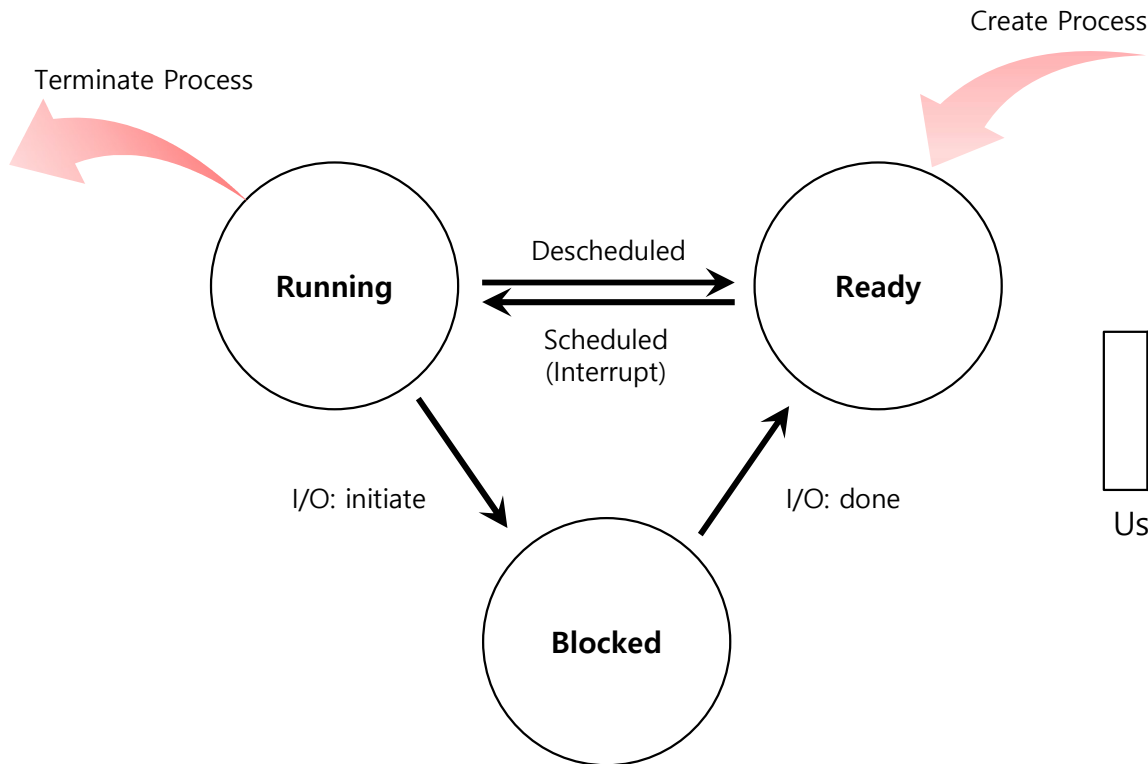
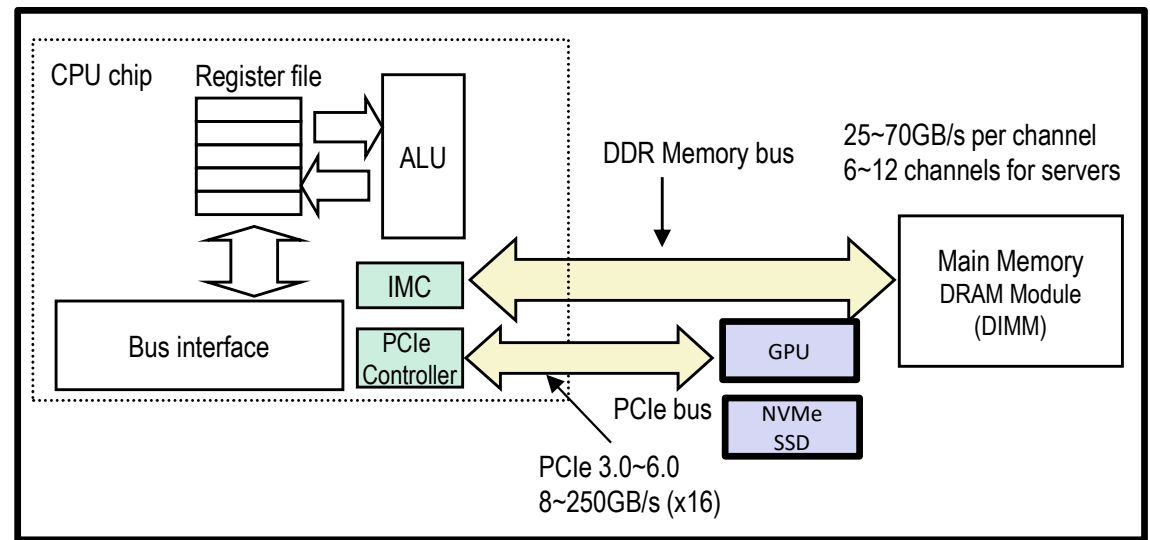
Contains **temporary data**
such as function parameters,
return addresses,
and local variables

Contains **memory**
that is **dynamically**
allocated during process runtime
(e.g., malloc())

Contains **global variables**

Recap)

Process State Transition & I/O Data Flow



Today

- **Process**

- `fork()`
- `exec()`
- `wait()`

- **Thread**

- **Concurrency Problem**

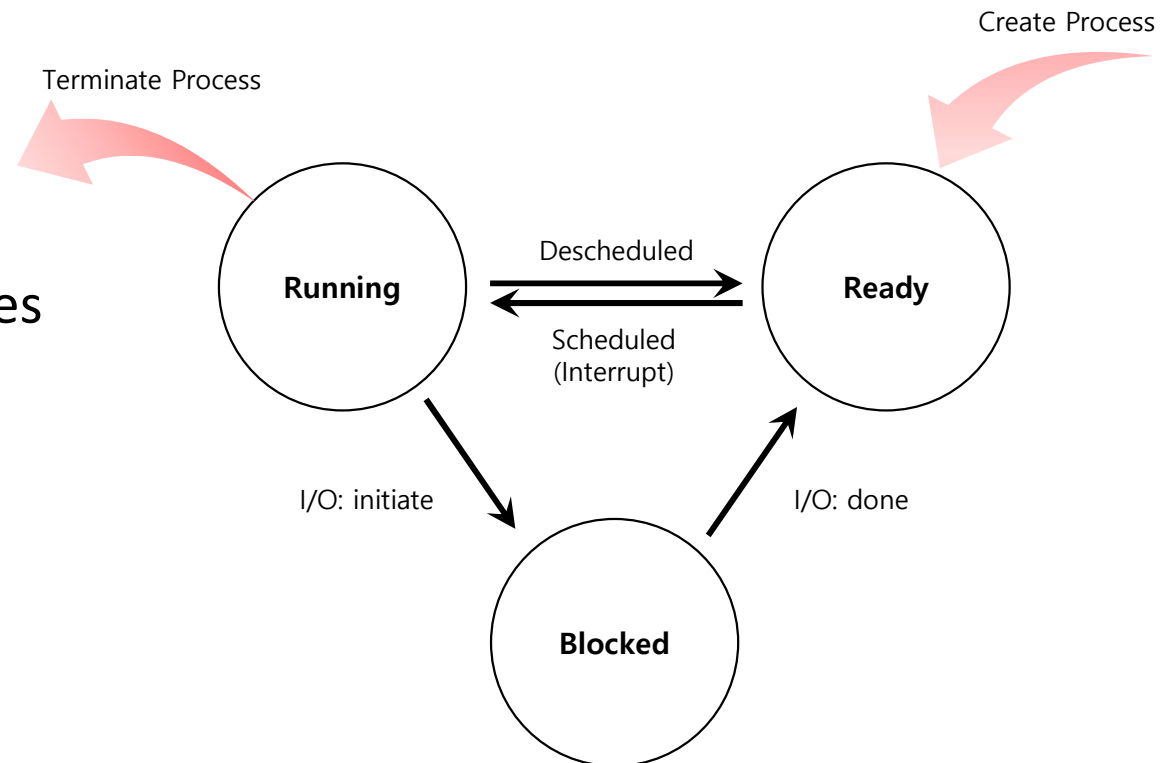
Data structures

■ PCB(Process Control Block)

- A C-structure that contains information **about each process**.
- **Register context:** a set of registers that define the state of a process

■ Process list

- Ready processes
- Blocked processes
- Current running processes



Example) The xv6 kernel Proc Structure

```
// the information xv6 tracks about each process
// including its register context and state
struct proc {
    char *mem;           // Start of process memory
    uint sz;             // Size of process memory
    char *kstack;        // Bottom of kernel stack
                        // for this process

    enum proc_state state; // Process state
    int pid;              // Process ID
    struct proc *parent;  // Parent process
    void *chan;           // If non-zero, sleeping on chan
    int killed;           // If non-zero, have been killed
    struct file *ofile[NOFILE]; // Open files
    struct inode *cwd;    // Current directory
    struct context context; // Switch here to run process
    struct trapframe *tf; // Trap frame for the
                        // current interrupt
};
```

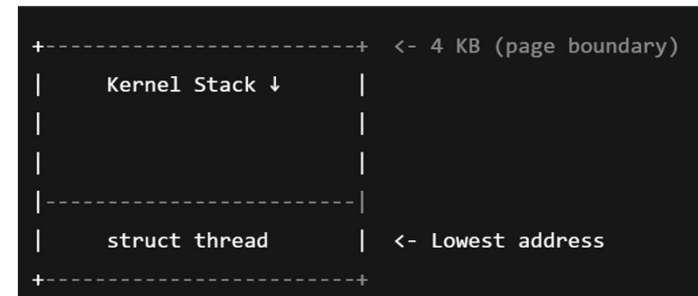
Example) Register Context in xv6

```
// the registers xv6 will save and restore
// to stop and subsequently restart a process
struct context {
    int eip;    // Index pointer register
    int esp;    // Stack pointer register
    int ebx;    // Called the base register
    int ecx;    // Called the counter register
    int edx;    // Called the data register
    int esi;    // Source index register
    int edi;    // Destination index register
    int ebp;    // Stack base pointer register
};

// the different states a process can be in
enum proc_state { UNUSED, EMBRYO, SLEEPING,
                  RUNNABLE, RUNNING, ZOMBIE };
```


Example)

thread structure in Pintos



```
struct thread{
    /* Owned by thread.c. */
    tid_t tid;                                /* Thread identifier. */
    enum thread_status status;                /* Thread state. */
    char name[16];                             /* Name (for debugging purposes). */
    uint8_t *stack;                           /* Saved stack pointer. */
    int priority;                              /* Priority. */
    struct list_elem allelem; /* List element for all threads list. */

    /* Shared between thread.c and synch.c. */
    struct list_elem elem; /* List element. */

#ifdef USERPROG
    /* Owned by userprog/process.c. */
    uint32_t *pagedir; /* Page directory. */
#endif

    /* Owned by thread.c. */
    unsigned magic; /* Detects stack overflow. */
};
```

Process Control Block in Linux

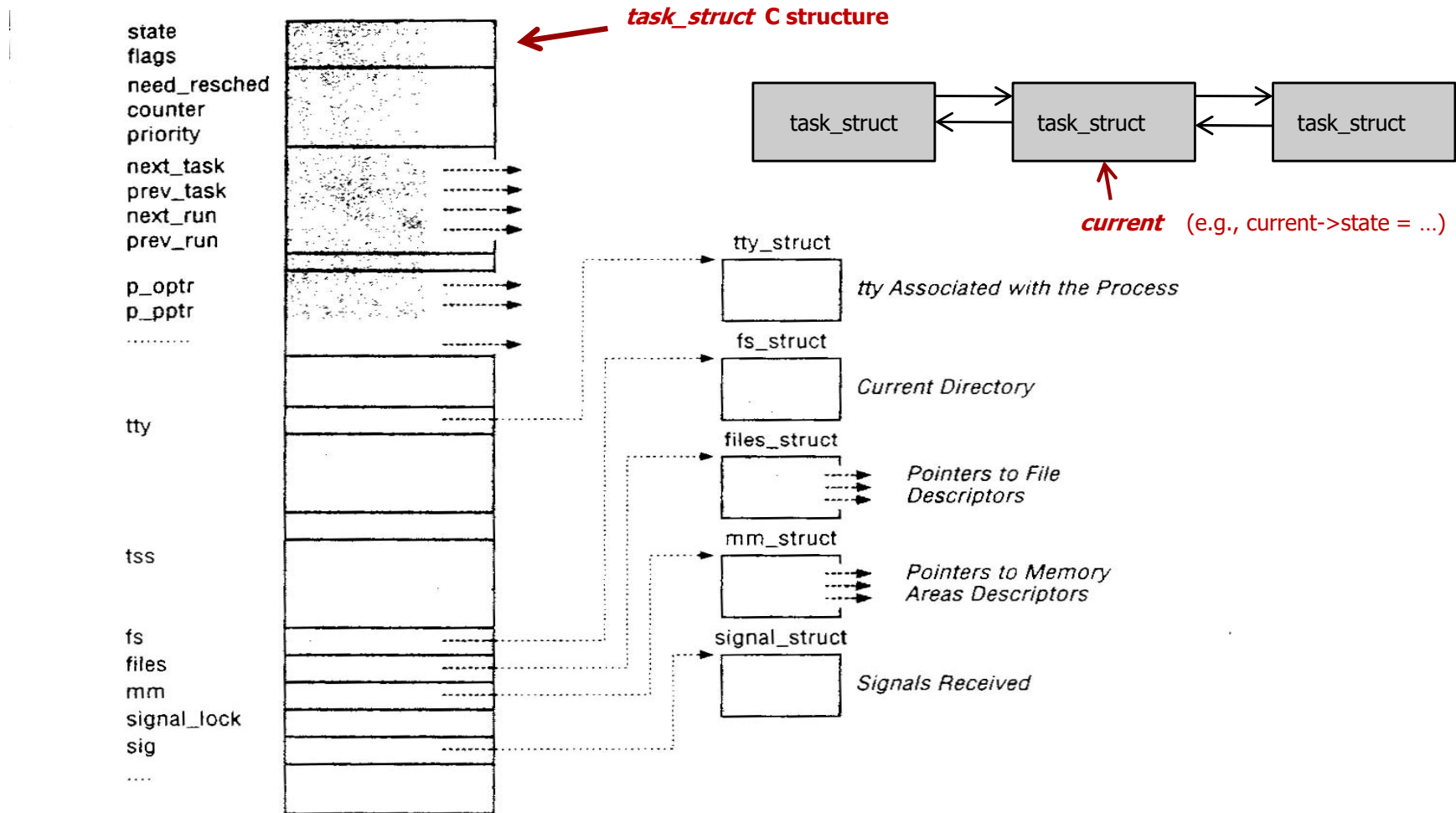


Figure 3-1. The Linux process descriptor

Process Control Block

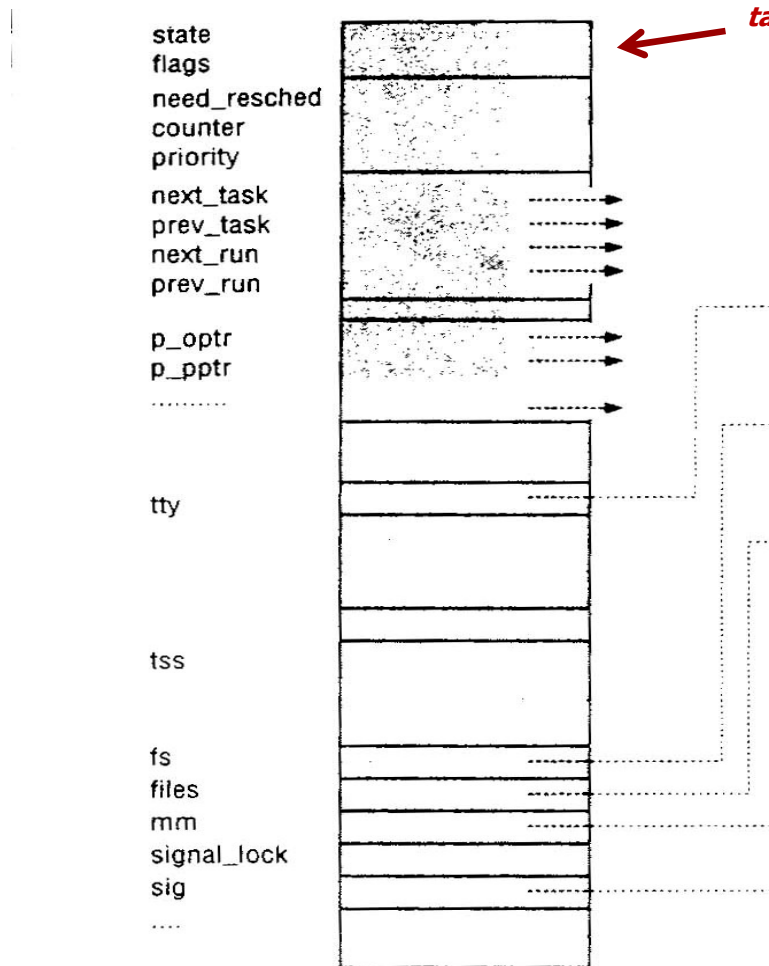


Figure 3-1. The Linux process descriptor

```

struct task_struct {
    /* ---- 1. 프로세스 기본 정보 ---- */
    pid_t pid;                /* 프로세스 ID */
    pid_t tgid;               /* 스레드 그룹 ID */
    long state;               /* 실행 상태 (RUNNING, SLEEPING 등) */
    unsigned int flags;       /* 커널 제어 플래그 */
    char comm[TASK_COMM_LEN]; /* 프로세스 이름 (최대 16자) */

    /* ---- 2. 스케줄링 정보 ---- */
    int prio;                 /* 현재 우선순위 */
    int static_prio;          /* 기본 우선순위 */
    unsigned int policy;      /* 스케줄링 정책 (CFS, FIFO, RR 등) */
    struct sched_entity se;    /* 스케줄러가 사용하는 실행 단위 정보 */
    struct list_head tasks;   /* 전체 프로세스 리스트 연결 */

    /* ---- 3. 부모 / 자식 관계 ---- */
    struct task_struct *parent; /* 부모 프로세스 */
    struct list_head children;  /* 자식 프로세스 리스트 */

    /* ---- 4. 메모리 관리 ---- */
    struct mm_struct *mm;      /* 유저 주소 공간 정보 */
    struct mm_struct *active_mm; /* 커널 스레드의 가상 메모리 */

    /* ---- 5. 파일 시스템 ---- */
    struct fs_struct *fs;      /* 현재 작업 디렉토리 / 루트 디렉토리 */
    struct files_struct *files; /* 열린 파일 테이블 */

    /* ---- 6. 시그널 / 인터럽트 ---- */
    struct signal_struct *signal; /* 시그널 상태 */
    struct sighand_struct *sighand; /* 시그널 핸들러 테이블 */

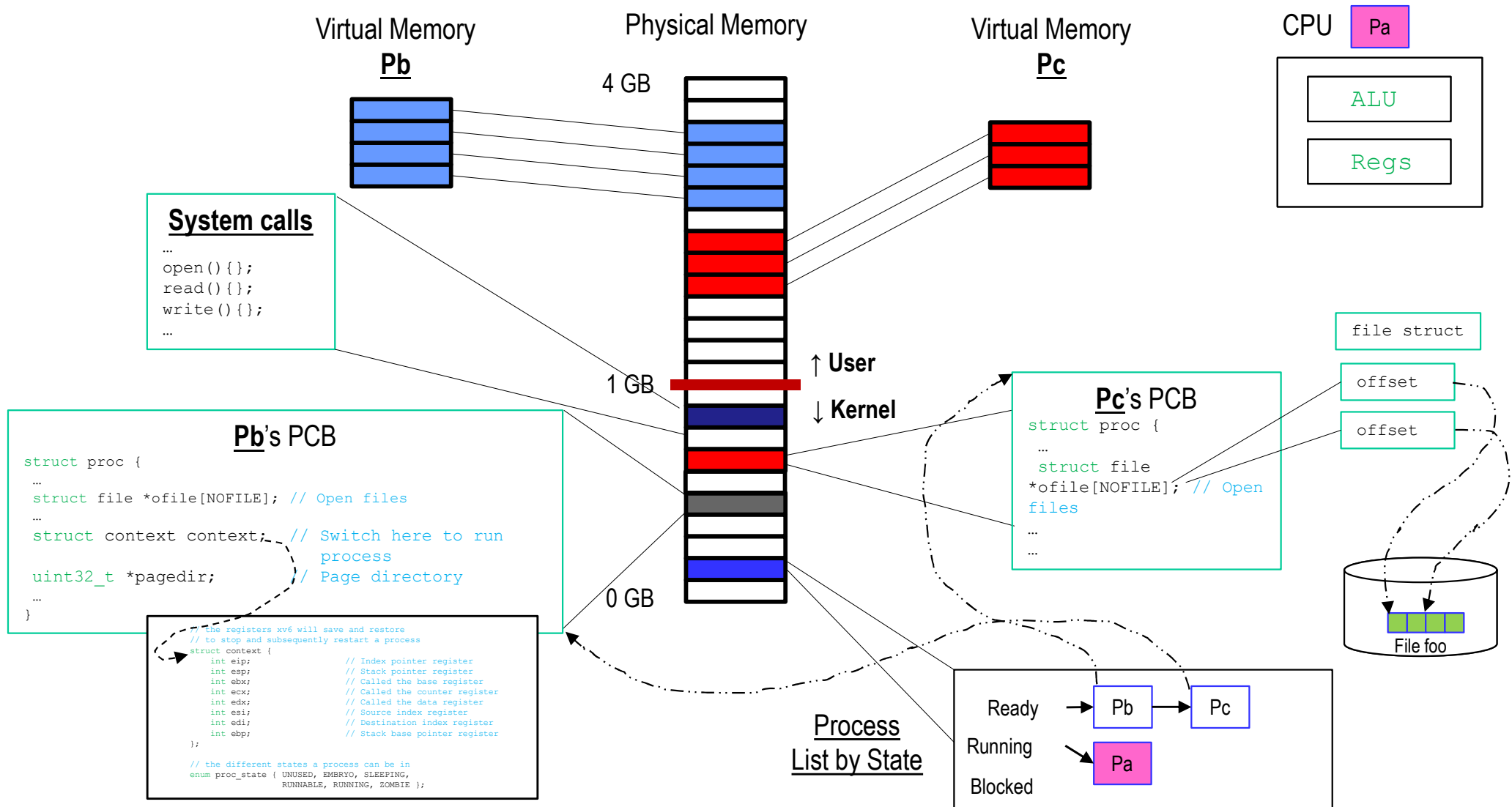
    /* ---- 7. 하드웨어 문맥 (레지스터 상태 등) ---- */
    struct thread_struct thread; /* CPU 레지스터, 스택 포인터, FPU 등 */

    /* ---- 8. 시간 및 통계 ---- */
    u64 start_time;           /* 시작 시각 */
    u64 utime;                /* 사용자 모드 실행 시간 */
    u64 stime;                /* 커널 모드 실행 시간 */

    /* ---- 9. 자격/보안 ---- */
    const struct cred *cred;   /* UID, GID, capabilities 등 보안 정보 */
};
    
```

가정:

- 물리적인 메모리 (Physical Memory) 크기 4GB
- 페이지 크기: 4KB (아래 그림에서 한 개 박스!)
- **가상메모리 (응용이 사용 가능한) 최대 16KB (그 이상 허용 불가!)**
- 가상메모리 -> 물리적인 메모리 매핑: **linear mapping** (1-1 매핑)
 - ✓ Directory 같은 구조는 필요 없음. 왜냐하면, linear mapping이므로 첫 페이지가 mapping되어 있는 물리적인 페이지 주소만 알고 있으면 됨
- 아래 그림에서 프로세스 가상 메모리 커널 부분은 설명의 용이를 위해 hide했음



Creating a child process

`fork()`



■ Create a child process

- Child process is allocated separate memory space from the process. The child process has the same memory contents as the parents.
- The child process has its own **registers**, and program counter register(**PC**).
- The newly created process becomes independent after it is created.
- For parent, `fork()` returns PID of child process; for child process, `fork()` returns 0.

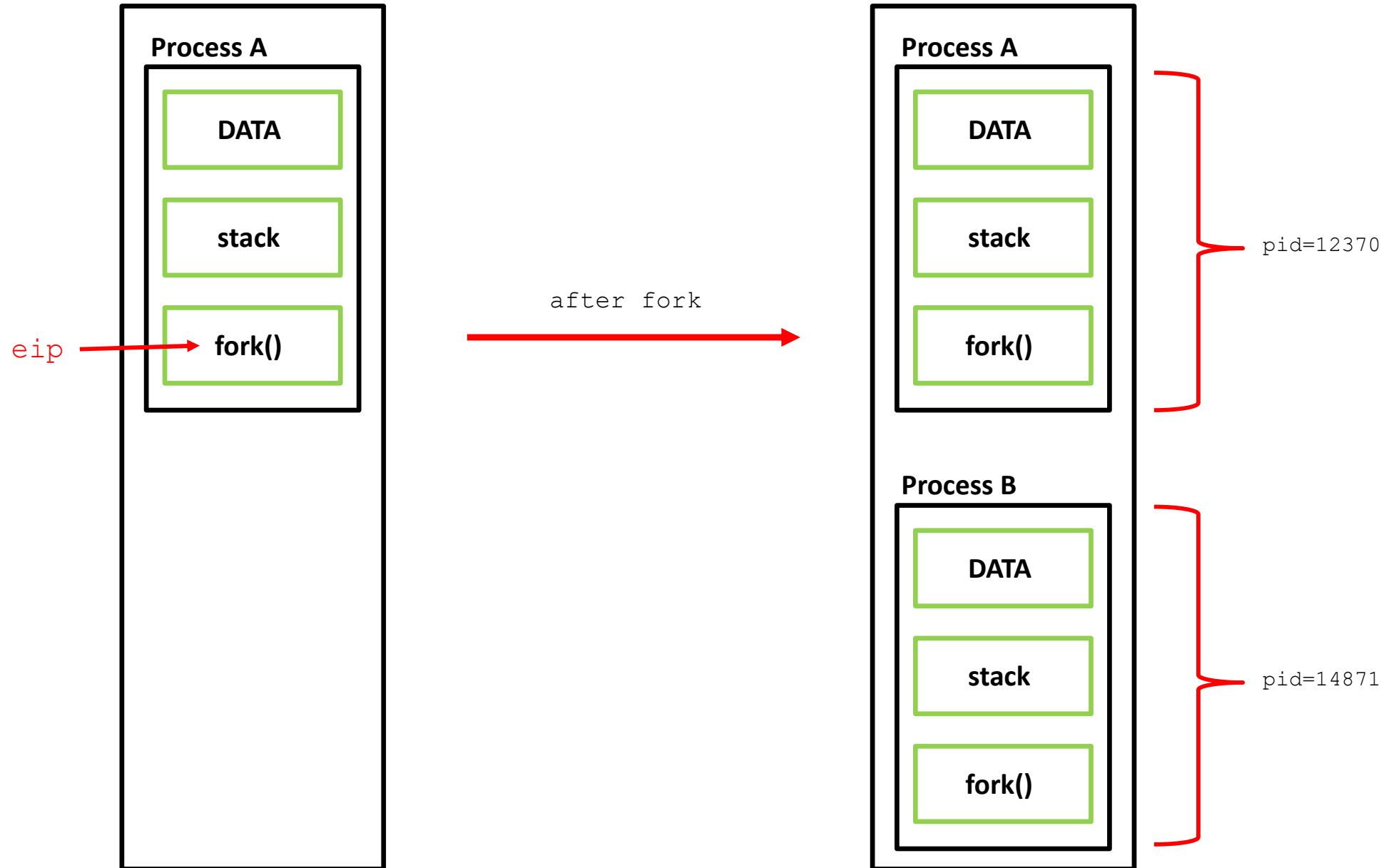
Usage of fork ()

p1.c

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main(int argc, char *argv[]){
    printf("hello world (pid:%d)\n", (int) getpid());
    int rc = fork();
    if (rc < 0) {    // fork failed; exit
        fprintf(stderr, "fork failed\n");
        exit(1);
    } else if (rc == 0) { // child (new process)
        printf("hello, I am child (pid:%d)\n", (int) getpid());
    } else {          // parent goes down this path (main)
        printf("hello, I am parent of %d (pid:%d)\n",
            rc, (int) getpid());
    }
    return 0;
}
```

fork



fork(): parent vs. child

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

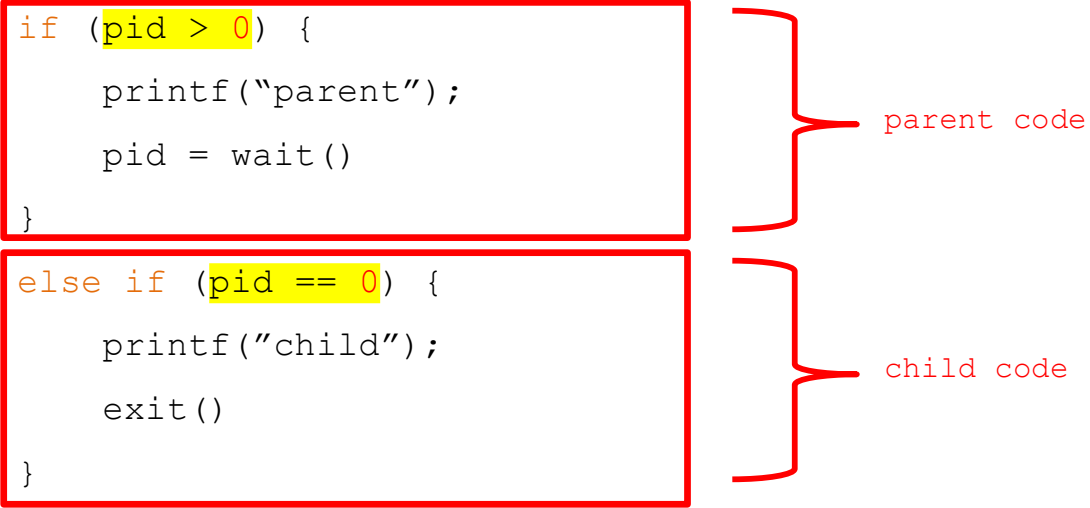
int main(int argc, char *argv[]){
    int pid = fork();

    if (pid > 0) {
        printf("parent");
        pid = wait()
    }

    else if (pid == 0) {
        printf("child");
        exit()
    }

    else {
        printf("fork error");
    }

    return 0;
}
```



The diagram illustrates the execution flow of the `fork()` function. It features two red rectangular boxes. The first box, labeled "parent code" with a red bracket, contains the code for the parent process: `if (pid > 0) { printf("parent"); pid = wait(); }`. The second box, labeled "child code" with a red bracket, contains the code for the child process: `else if (pid == 0) { printf("child"); exit(); }`. The `else` block for fork errors is not enclosed in a box.

Let's run it.

```
prompt> ./p1  
hello world (pid:29146)  
hello, I am parent of 29147 (pid:29146)  
hello, I am child (pid:29147)  
prompt>
```

or

```
prompt> ./p1  
hello world (pid:29146)  
hello, I am child (pid:29147)  
hello, I am parent of 29147 (pid:29146)  
prompt>
```

Create the dependency between the processes

`wait()`

- When the child process is created, `wait()` in the parent process won't return until the child has run and exited.
- The parent and the child does not have any dependency.
- In some cases, the application wants to enforce the order in which they are executed, e.g. the parent exits only after the child finishes.

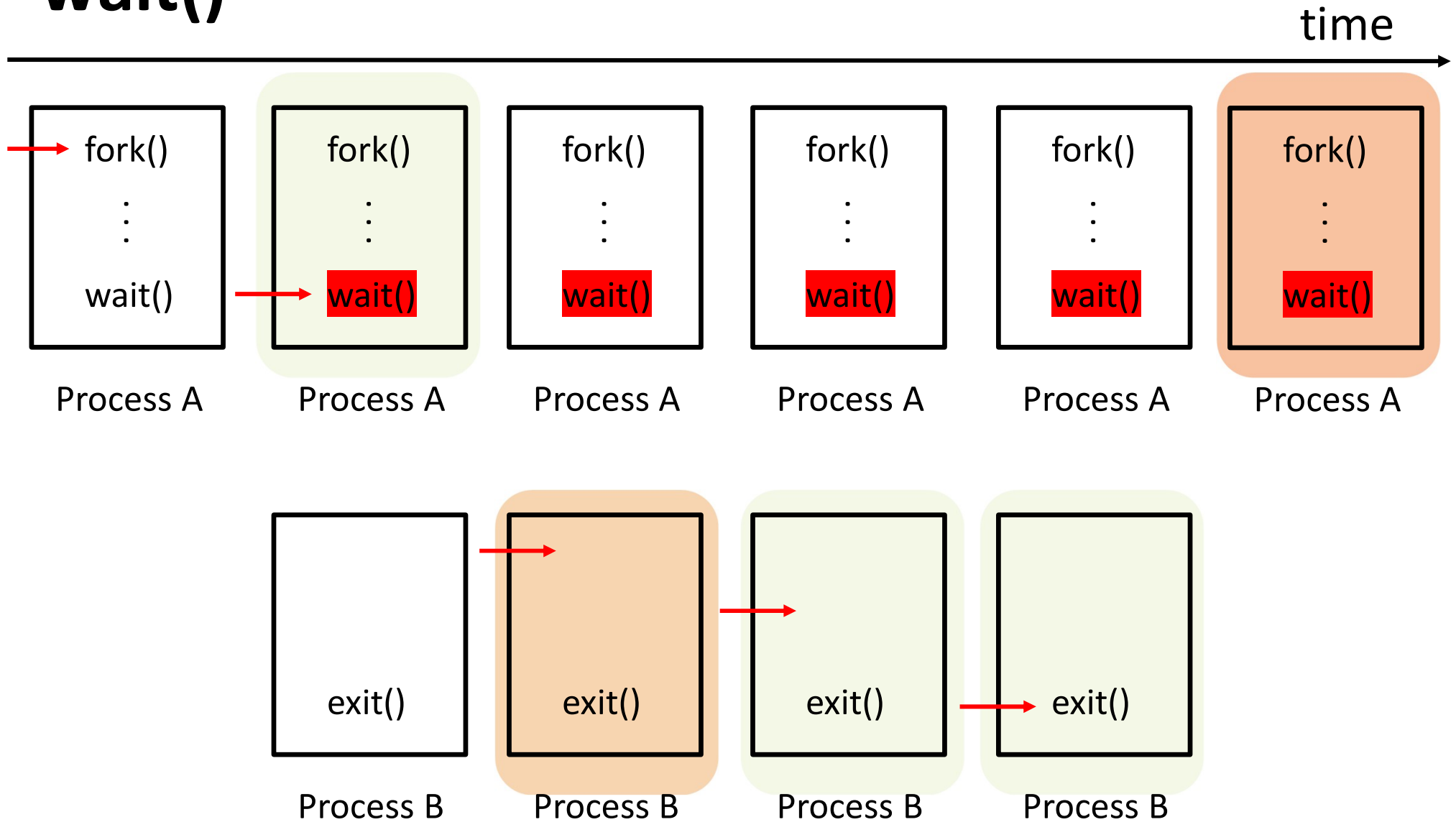
The usage of wait () System Call

p2.c

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main(int argc, char *argv[]){
    printf("hello world (pid:%d)\n", (int) getpid());
    int rc = fork();
    if (rc < 0) {                // fork failed; exit
        fprintf(stderr, "fork failed\n");
        exit(1);
    } else if (rc == 0) { // child (new process)
        printf("hello, I am child (pid:%d)\n", (int) getpid());
    } else {                    // parent goes down this path (main)
        int wc = wait(NULL);
        printf("hello, I am parent of %d (wc:%d) (pid:%d)\n",
            rc, wc, (int) getpid());
    }
    return 0;
}
```

wait()



The wait() System Call (Cont.)

Result (Deterministic)

```
prompt> ./p2
hello world (pid:29266)
hello, I am child (pid:29267)
hello, I am parent of 29267 (wc:29267) (pid:29266)
prompt>
```

Does fork() immediately copy the entire process heap in Linux?

A `fork()` system call clones a child process from the running process. The two processes are

identical except for the PID.

Naturally, if the processes are just reading from their heaps rather than writing to it, copying the heap would be a huge waste of memory.

Is the entire process heap copied? Is it optimized in a way that only writing triggers a heap copy?

The Linux kernel does implement Copy-on-Write when `fork()` is called. When the syscall is executed, the pages that the parent and child share are marked read-only.

If a write is performed on the read-only page, it is then copied, as the memory is no longer identical between the two processes. Therefore, if only read-operations are being performed, the pages will not be copied at all.

[Does fork\(\) immediately copy the entire process heap in Linux? - Unix & Linux Stack Exchange](#)

Running a new program

`exec()`

- **The caller wants to run a program that is different from the caller itself.**
 - Launch an editor
 - `% ls -l`
- **OS needs to load a new binary image, initialize a new stack, initialize a new heap for the new program.**
- **Two parameters**
 - The name of the binary file
 - The array of arguments

```
char *argv[3];

argv[0] = "echo";
argv[1] = "hello";
argv[2] = 0;
exec("/bin/echo", argv);
printf("exec error\n");
```


Usage of exec ()

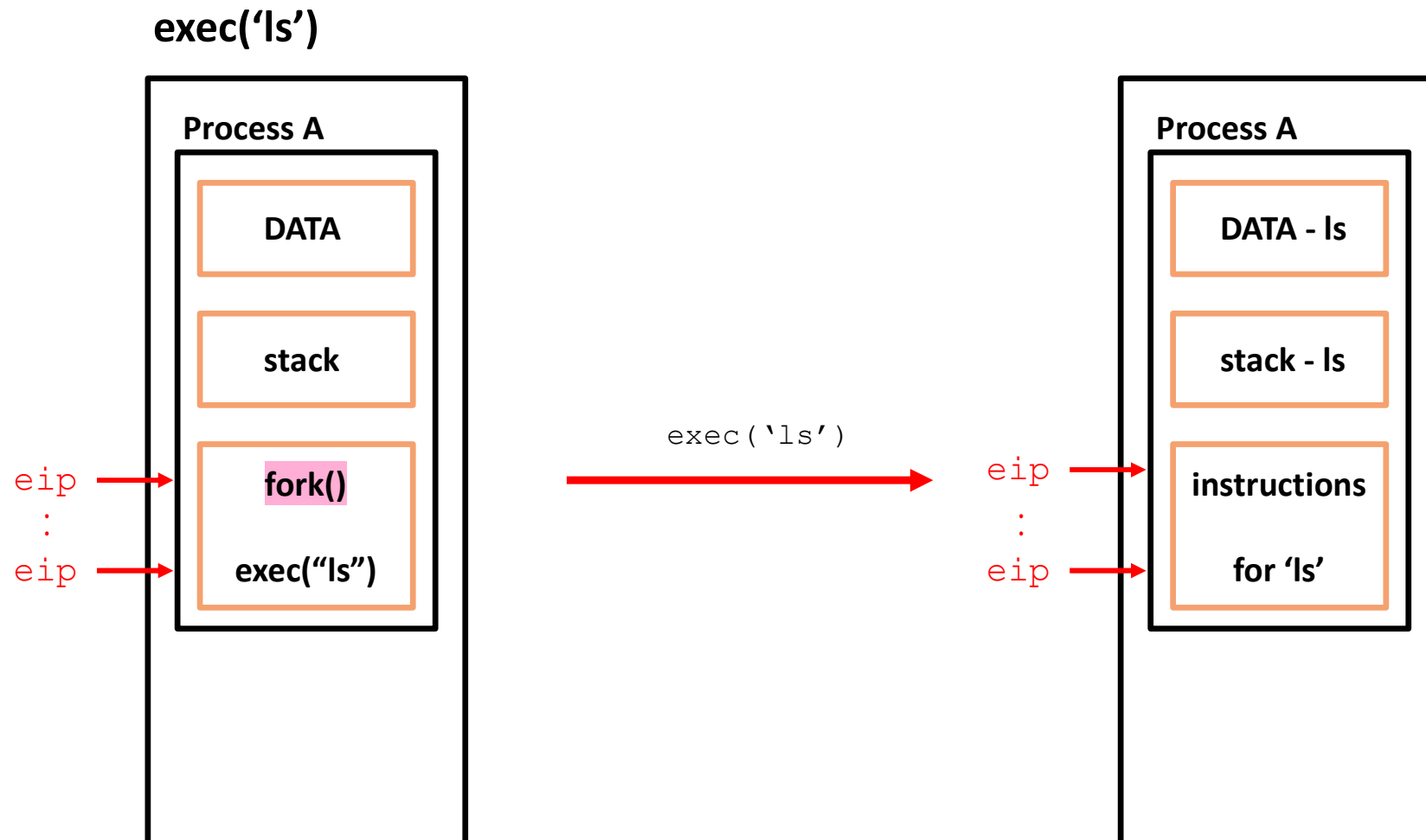
p3.c

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <string.h>
#include <sys/wait.h>

int main(int argc, char *argv[]){
    printf("hello world (pid:%d)\n", (int) getpid());
    int rc = fork();
    if (rc < 0) {                // fork failed; exit
        fprintf(stderr, "fork failed\n");
        exit(1);
    } else if (rc == 0) {        // child (new process)
        printf("hello, I am child (pid:%d)\n", (int) getpid());
        char *myargs[3];
        myargs[0] = strdup("wc");                // program: "wc" (word count)
        myargs[1] = strdup("p3.c");              // argument: file to count
        myargs[2] = NULL;                        // marks end of array
        execvp(myargs[0], myargs); // runs word count
        printf("this shouldn't print out");
    } else {                        // parent goes down this path (main)
        int wc = wait(NULL);
        printf("hello, I am parent of %d (wc:%d) (pid:%d)\n",
            rc, wc, (int) getpid());
    }
    return 0;
}
```

When `exec()` is called,...

- ▣ Replace the existing contents of the memory with the new memory contents from the new binary file.
- ▣ `exec()` does not return. It starts to execute the new program.



Usage of `exec()`

Result

```
prompt> ./p3
hello world (pid:29383)
hello, I am child (pid:29384)
29 107 1030 p3.c
hello, I am parent of 29384 (wc:29384) (pid:29383)
prompt>
```

Why separating `fork()` and `exec()`?

- Why don't we just use something like “`forkandexec("ls", "ls -l")`”?
- Via separating `fork()` and `exec()`, we can manipulate various settings just before executing a new program and make the IO redirection and pipe possible.

- IO redirection

```
% cat w3.c > newfile.txt
```

- Pipe

```
% echo hello world | wc
```

Today

- **Process**

- `fork()`
- `exec()`
- `wait()`

- **Thread**

- **Concurrency Problem**

What is a Thread? — Flow of Instructions

■ Concept

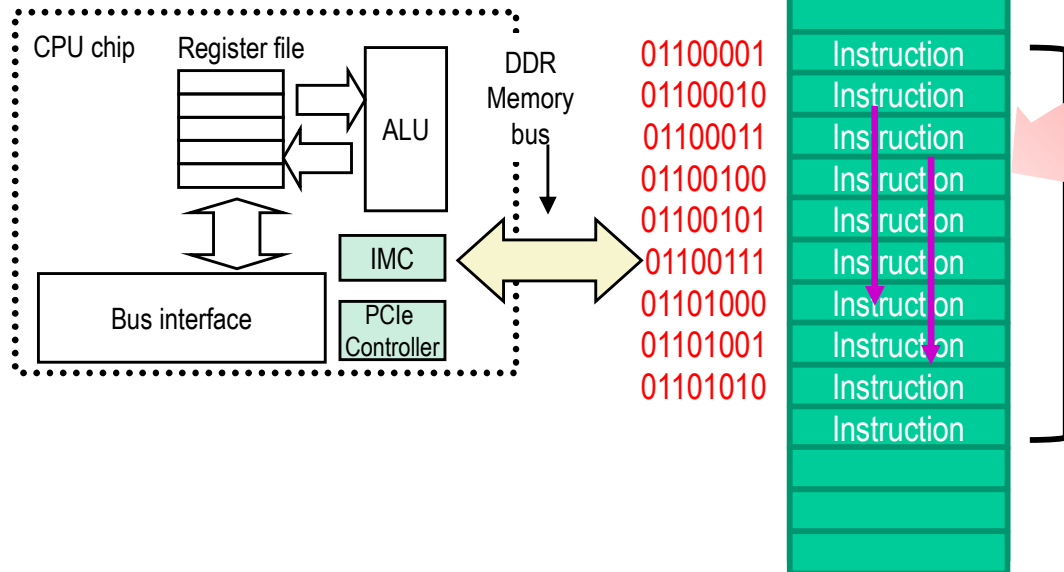
- A thread represents a *sequence of instructions* executed by the CPU.
- It defines how a program's instructions flow through time — the path of execution.
- Each program may have one or more threads, each following its own control flow.

■ Structure

- Program: A static set of instructions (code).
- Thread: The *active execution flow* of those instructions.
- Each thread has:
 - **Program Counter (PC):** tracks the next instruction to execute
 - **Stack:** stores function calls and local variables
 - **Registers:** hold temporary computation data

Example

Two Threads
=
Two Execution
Flows



```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include "common.h"
4
5 volatile int counter = 0;
6 int loops;
7
8 void *worker(void *arg) {
9     int i;
10     for (i = 0; i < loops; i++) {
11         counter++;
12     }
13     return NULL;
14 }
15
16 int
17 main(int argc, char *argv[])
18 {
19     if (argc != 2) {
20         fprintf(stderr, "usage: threads
21             <value>\n");
22         exit(1);
23     }
24     loops = atoi(argv[1]);
25     pthread_t p1, p2;
26     printf("Initial value : %d\n", counter);
27     Pthread_create(&p1, NULL, worker, NULL);
28     Pthread_create(&p2, NULL, worker, NULL);
29     Pthread_join(p1, NULL);
30     Pthread_join(p2, NULL);
31     printf("Final value : %d\n", counter);
32     return 0;
33 }
```

- A single thread executes these instructions in order, one after another.
- **Multiple threads** can run **different instruction/execution flows** within the **same program**.

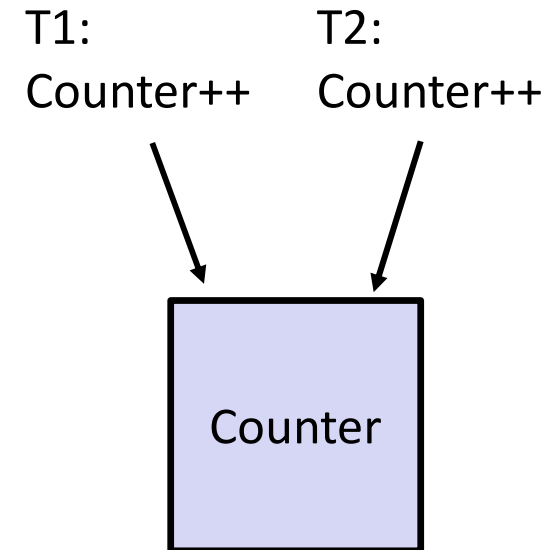
Concurrency Problem

- The OS handles many processes simultaneously by rapidly switching between them.
- This time-sharing creates the illusion that programs run at the same time.
- In **multi-threaded programs**, multiple threads within one process run concurrently.
- Such parallel execution can cause concurrency issues like **race conditions**.

Concurrency Example

■ A Multi-threaded Program (thread.c)

```
1      #include <stdio.h>
2      #include <stdlib.h>
3      #include "common.h"
4
5      volatile int counter = 0;
6      int loops;
7
8      void *worker(void *arg) {
9          int i;
10         for (i = 0; i < loops; i++) {
11             counter++;
12         }
13         return NULL;
14     }
15
16     int
17     main(int argc, char *argv[])
18     {
19         if (argc != 2) {
20             fprintf(stderr, "usage: threads <value>\n");
21             exit(1);
22         }
```



Concurrency Example (Cont.)

```
23         loops = atoi(argv[1]);
24         pthread_t p1, p2;
25         printf("Initial value : %d\n", counter);
26
27         Pthread_create(&p1, NULL, worker, NULL);
28         Pthread_create(&p2, NULL, worker, NULL);
29         Pthread_join(p1, NULL);
30         Pthread_join(p2, NULL);
31         printf("Final value : %d\n", counter);
32         return 0;
33     }
```

- The main program creates **two threads**.
 - Thread: a function running within the same memory space.
 - Each thread start running in a routine called `worker()`.
 - `worker()`: increments a counter

Declaring a variable with the **volatile** keyword

■ **volatile int counter = 0;**

- In C, declaring a variable as volatile tells the compiler not to optimize away memory accesses for that variable.

■ **When can values change unexpectedly?**

- Interrupt Service Routines (ISRs) that update variables asynchronously
- Shared variables in multi-threaded or multi-core programs

```
volatile int flag = 0;

void interrupt_handler() {
    flag = 1; // Changed inside ISR
}

int main() {
    while (flag == 0) {
        // Because of volatile, flag is
        // always read from memory
    } // Loop exits once flag becomes 1
}
```

Without volatile, the compiler might optimize the loop into an infinite loop, assuming flag never changes.

Example

C

```
int x;  
while (x == 0) { }
```

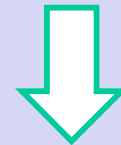


asm

```
mov eax, DWORD PTR [x]  
.L1:  
cmp eax, 0  
je .L1
```

C

```
volatile int x;  
while (x == 0) { }
```



asm

```
.L1:  
mov eax, DWORD PTR [x]  
    ; always reload from memory  
cmp eax, 0  
je .L1
```

Concurrency Example (Cont.)

- **loops** determines how many times each of the two workers will increment the shared counter in a loop.

- `loops: 1000.`

```
prompt> gcc -o thread thread.c -Wall -pthread
prompt> ./thread 1000
Initial value : 0
Final value : 2000
```

- `loops: 100000.`

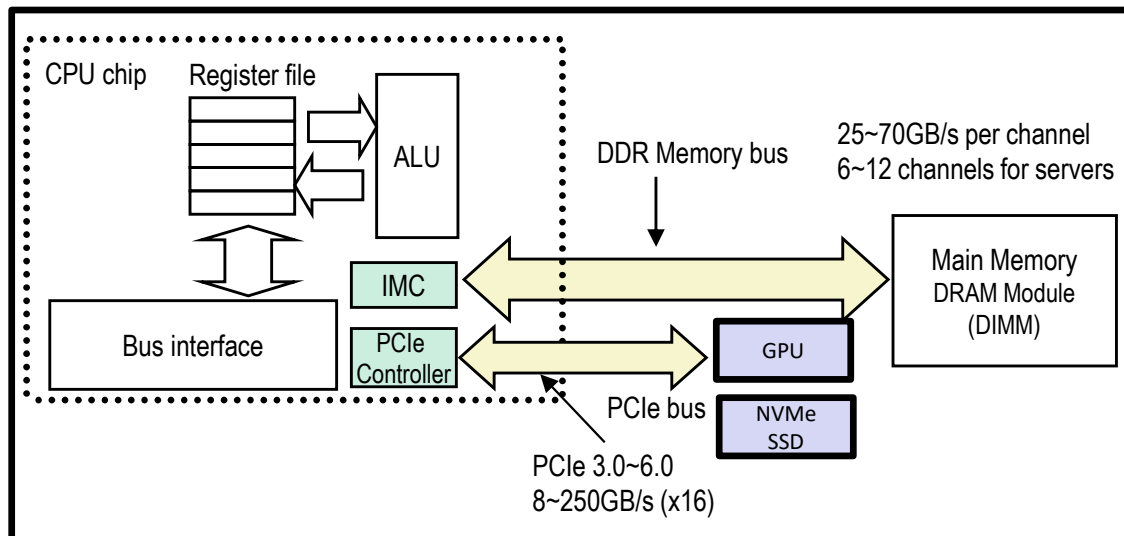
```
prompt> ./thread 100000
Initial value : 0
Final value : 143012 // huh??
prompt> ./thread 100000
Initial value : 0
Final value : 137298 // what the??
```

Why is this happening?

- **Increment a shared counter → take three instructions.**

- ① Load the value of the counter from memory into register.
- ② Increment it
- ③ Store it back into memory

- **These three instructions do not execute *atomically*.**
→ **Concurrency problem happens.**



Assembly Code for Counter Loop

C code for counter loop in thread i

```
for (i = 0; i < loops; i++)
    counter++;
```

Asm code for thread i

H_i : Head

L_i : Load cnt

U_i : Update cnt

S_i : Store cnt

T_i : Tail

```
movq    (%rdi), %rcx
testq   %rcx, %rcx
jle     .L2
movl    $0, %rax
```

.L3:

```
movq    cnt(%rip), %rdx
addq    $1, %rdx
movq    %rdx, cnt(%rip)
```

```
addq    $1, %rax
cmpq    %rcx, %rax
jne     .L3
```

.L2:

1. %rdi가 가리키는 메모리 주소에서
8바이트 값을 읽어 %rcx에 저장
2. %rcx == 0 인지 확인 (ZF 설정)
3. %rcx <= 0이면 .L2로 점프 (<= 비교는 signed 기준)
4. 그렇지 않으면 %rax를 0으로 초기화

.L3:

5. 전역 변수 cnt 값을 %rdx에 로드
(RIP-relative addressing)
6. %rdx에 1을 더함
7. 다시 cnt에 %rdx를 저장 (cnt = cnt + 1)

8. 루프 인덱스 %rax += 1
9. %rax와 %rcx를 비교
10. 같지 않으면 다시 .L3으로 루프

11. 루프 종료 지점

Concurrent Execution

- **Key idea:** In general, any sequentially consistent interleaving is possible, but some give an unexpected result!

- I_i denotes that thread i executes instruction I
- $\%rdx_i$ is the content of $\%rdx$ in thread i 's context

i (thread)	$instr_i$	$\%rdx_1$	$\%rdx_2$	cnt
1	H_1	-	-	0
1	L_1	0	-	0
1	U_1	1	-	0
1	S_1	1	-	1
2	H_2	-	-	1
2	L_2	-	1	1
2	U_2	-	2	1
2	S_2	-	2	2
2	T_2	-	2	2
1	T_1	1	-	2



Thread 1
critical section



Thread 2
critical section

OK

Concurrent Execution (cont)

- **Incorrect ordering: two threads increment the counter, but the result is 1 instead of 2**

i (thread)	instr _i	%rdx ₁	%rdx ₂	cnt
1	H ₁	-	-	0
1	L ₁	0	-	0
1	U ₁	1	-	0
2	H ₂	-	-	0
2	L ₂	-	0	0
1	S ₁	1	-	1
1	T ₁	1	-	1
2	U ₂	-	1	1
2	S ₂	-	1	1
2	T ₂	-	1	1

 Thread 1
critical section

 Thread 2
critical section

Oops!

Concurrent Execution (cont)

■ How about this ordering?

i (thread)	instr _i	%rdx ₁	%rdx ₂	cnt
1	H ₁			0
1	L ₁	0		
2	H ₂			
2	L ₂		0	
2	U ₂		1	
2	S ₂		1	1
1	U ₁	1		
1	S ₁	1		1
1	T ₁			1
2	T ₂			1

Oops!

OS Basics and Process: Summary

■ OS = Virtualization + Resource Management

- Transforms hardware (CPU, memory, disk) into virtual and manageable resources
- Makes programs run efficiently, safely, and concurrently

■ Process vs. Thread

- Process: A program in execution (code/data, heap, stack, registers)
 - States: Running, Ready, Blocked
- Thread: A *sequence of instructions* executed by the CPU
- Threads within the same process share its code, data, heap, and file resources.

■ Concurrency Problem

- Multiple threads can cause race conditions (counter++)